

Name: Jiahui Liu (Johnny)

Course: COMP-258, Data Structures & Frameworks

Program: Business Information Systems

Lab: Linked List Data Structures – Algorithm Efficiency

My Answers to Part 2 (Part 1 is completed in NetBeans)

Question: Provide an analysis of the two algorithms. Which has the worst case run time? Explain your answer using an example. Provide an approximate number of operations, for both methods, for a collection of size n . Provide a graph of the comparison. Place this in an “analysis” folder in your project.

Answer:

- **Algorithm Categorization & Overview:** Both `ArrayManager.removeAllOccurrences` and `LinkedList.removeAllOccurrences` belong to the linear iteration category of searching and filtering algorithms. Both implementations utilize a single-pass traversal pattern to identify and eliminate target matches.
- **Worst-Case Runtime Analysis:**
 - **ArrayManager Worst-Case Time Complexity:** $O(n)$
 - **LinkedList Worst-Case Time Complexity:** $O(n)$Both algorithms exhibit identical worst-case Big-O runtime asymptotic efficiency of $O(n)$ where n represents the number of elements in the collection. Neither algorithm is asymptotically "worse" than the other, though internal CPU dynamic operations differ.
 - **ArrayManager ($O(n)$):** Using a two-pointer approach (readIndex and writeIndex), the algorithm iterates through the array of size n exactly once. Every element is inspected once, and non-matching values are overwritten in-place, resulting in n comparisons and at most n assignments. A secondary loop zeroes out trailing unused capacity, yielding approximately $2n$ total operation steps.
 - **LinkedList ($O(n)$):** The algorithm traverses the node dynamic links from head to tail. Every node is visited once to check its inner data field (n comparisons). When a match occurs, pointer references are updated directly without shifting memory blocks, yielding approximately $1n$ to $1.5n$ total operation steps.

- **Worst-Case Example Scenario:** Considering a collection where EVERY element matches the target value o (e.g., collection = {4, 4, 4, 4, 4} where $n = 5$, and target $o = 4$):

- **ArrayManager Execution Trace:**

- Performs 5 array access checks and element comparisons against 4.
- Writes 0 elements to writeIndex because every element matches target.
- Runs cleanup loop 5 times to zero out dead slots.
- Total elementary operations ≈ 10 operations ($\sim 2n$).

- **LinkedList Execution Trace:**

- Performs 5 equality comparisons in the initial head-cleanup loop.
- Advances head reference 'items' 5 times until list reaches null.
- Traverses zero remaining nodes.
- Total elementary operations ≈ 5 operations ($\sim 1n$).

- **Approximate Operation Count Table (Dataset Size n):**

Operation Category	ArrayManager	LinkedList
Equality Comparisons	n element equality checks	n node data checks
Data Shifts/Overwrites	Up to n array assignments	0 memory shifts
Reference/Pointer Updates	0 pointer updates	Up to n reference updates (nextNode = nextNode.nextNode)
Cleanup/Nulling Loops	$n - \text{count}$ steps	0 cleanup steps
Total Operation Estimate	$\sim 2n$ steps	$\sim 1n$ steps
Big-O Asymptotic Complexity	$O(n)$	$O(n)$

- **Graph of Operation Count Comparison:**

